📄 PHASE 3 – Backend Schema & Database Design

Project: Prism IQ

Version: 1.0

Database: PostgreSQL

ORM: SQLAlchemy 2.0

Migration Tool: Alembic

Status: Design Freeze (Draft)

---

1. Database Design Philosophy

Prism IQ is **not** a simple CRUD application.

It is a **pipeline-driven analytics platform**.

Therefore, the database should **only store metadata**, while heavy artifacts (datasets, reports, trained models) should remain in the file system.

Why?

Imagine a user uploads:

```
150 MB CSV
```

Storing that inside PostgreSQL is a bad idea.

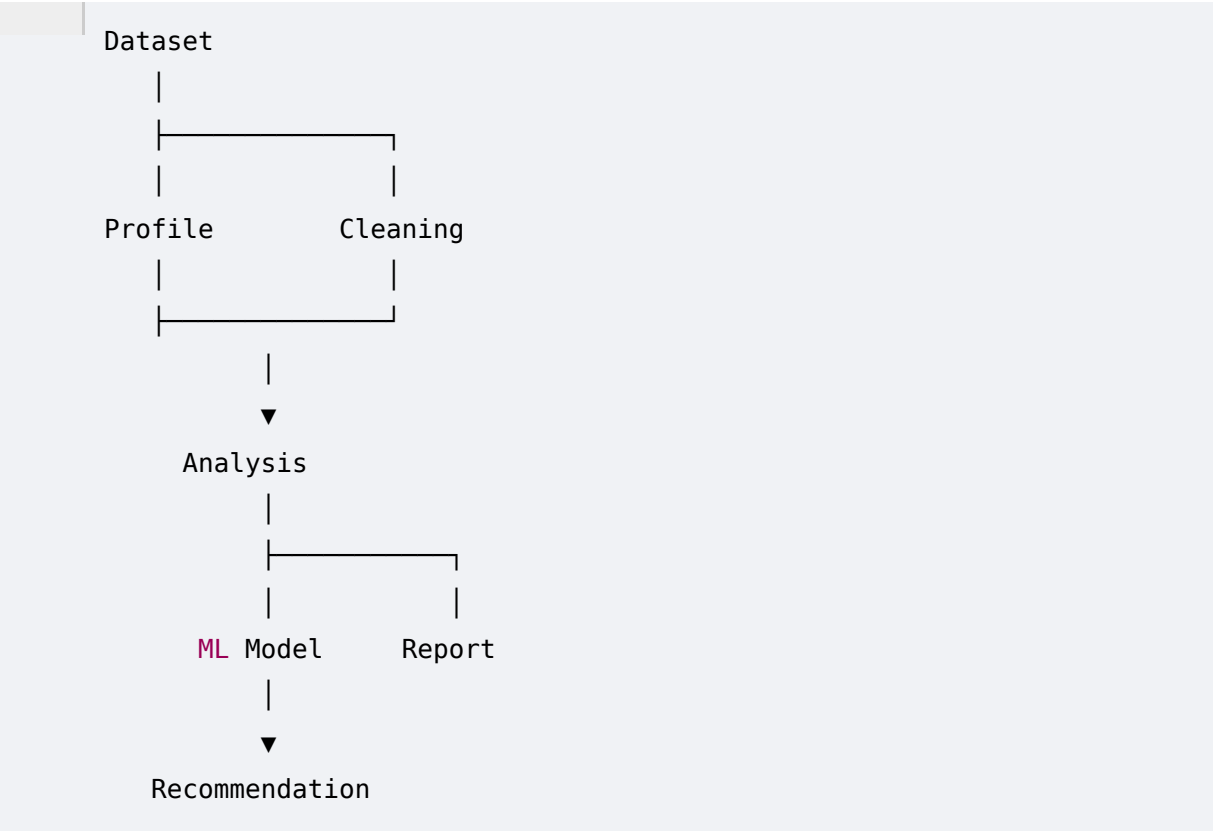
Instead:

```
CSV

↓

Filesystem

↓

Metadata → PostgreSQL
```

Industry products (Power BI, Tableau Prep, Databricks, etc.) follow similar principles.

---

2. High-Level ERD



Everything revolves around **Dataset**.

Dataset is our root entity.

---

3. Core Database Tables

For MVP I recommend only **8 tables**.

Not 20.

Not 40.

Just enough.

1️⃣ datasets

The heart of the system.

Stores uploaded dataset metadata.

Columns

```
id UUID PRIMARY KEY

dataset_name

original_filename
```

```
dataset_type

source_type

file_path

file_size

rows

columns
```

Status Values

```
UPLOADED

PROFILED

CLEANED

ANALYZED

TRAINED

COMPLETED

FAILED
```

Why no CSV here?

Because

```
CSV

↓

Filesystem

↓

Metadata in DB
```

2️⃣ data_profiles

Stores automatic profiling results.

Relationship

```
Dataset

1

↓

1

Profile
```

Columns

```
id

dataset_id

missing_values

duplicate_rows

memory_usage

column_summary (JSONB)

quality_score

created_at
```

Why JSONB?

Every dataset has different columns.

Instead of

```
column_1

column_2
```

```
column_3
```

Store

```
JSON
```

Much more scalable.

3️⃣ cleaning_jobs

Stores cleaning history.

```
id

dataset_id

cleaning_summary JSONB

rows_removed

duplicates_removed

missing_handled

outliers_handled

cleaned_file_path

created_at
```

One dataset can be cleaned multiple times.

Therefore:

```
Dataset

1

↓

N
```

```
Cleaning Jobs
```

4️⃣ analyses

Stores EDA metadata.

```
id

dataset_id

analysis_type

generated_charts JSONB

correlations JSONB

summary JSONB

created_at
```

Notice

Charts themselves are NOT stored.

Only metadata.

5️⃣ ml_models

Stores ML training runs.

Columns

```
id

dataset_id

target_column

problem_type

best_model

accuracy
```

```
precision

recall

f1_score
```

One dataset

↓

Many models

Relationship

```
Dataset

1

↓

N

Models
```

6️⃣ recommendations

Stores AI-generated recommendations.

```
id

dataset_id

recommendation_type

priority

title

description

confidence_score
```

```
created_at
```

Example

```
High

↓

Increase Electronics Inventory
```

7️⃣ reports

Stores generated reports.

```
id

dataset_id

report_type

report_path

generated_at
```

Example

```
Executive Report

Business Report

Summary Report
```

8️⃣ api_import_logs

Only for API/URL imports.

```
id

dataset_id

source_url
```

```
api_name

response_status

import_time
```

Useful for debugging.

---

4. Entity Relationships

```
Dataset
  |
  ├── 1 Profile
  |
  ├── N Cleaning Jobs
  |
  ├── N Analyses
  |
  ├── N ML Models
```

Very clean.

Very scalable.

---

5. Database Relationship Diagram

```
datasets
  |
  ├──────── data_profiles
```

```
      |
      ├──────── cleaning_jobs
      |
      ├──────── analyses
      |
      ├──────── ml models
```

No circular dependency.

Everything references Dataset.

---

6. UUID Strategy

Every table uses

```
UUID
```

instead of integer IDs.

Reason

Future cloud deployment.

Safer URLs.

No predictable IDs.

---

7. File Storage

```
uploads/
raw/
↓
dataset.csv
```

```
uploads/
cleaned/
```

```
↓

dataset_cleaned.csv
```

```
reports/

↓

executive_report.pdf
```

```
models/

↓

best_model.pkl
```

Database stores only paths.

---

8. JSONB Usage

Instead of creating 200 columns

Use PostgreSQL JSONB.

Example

```
{
  "Age": {
    "missing": 15,
    "dtype": "int64",
    "unique": 32
  },
  "Salary": {
    "missing": 5,
    "mean": 52000
  }
}
```

JSONB is perfect for variable dataset structures.

---

9. Indexing Strategy

Create indexes on

```
dataset_name

uploaded_at

status

dataset_type

target_column

problem_type
```

This will keep dashboard queries fast.

---

10. Constraints

Every FK

```
ON DELETE CASCADE
```

Meaning

Delete Dataset

↓

Delete everything related automatically.

---

11. Enum Types

Instead of VARCHAR.

Example

```
DatasetStatus

UPLOADED

PROFILED

CLEANED

ANALYZED
```

```
    TRAINED

    COMPLETED

    FAILED
```

ProblemType

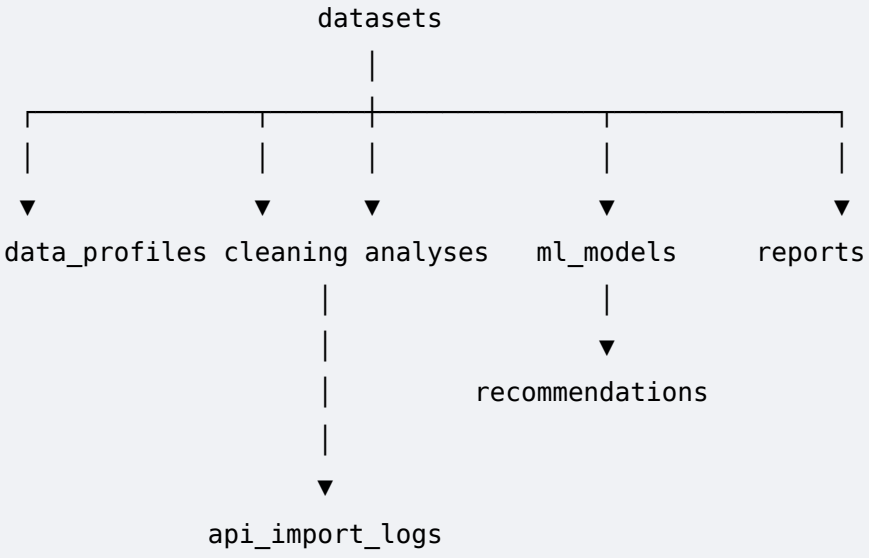
```
    REGRESSION

    CLASSIFICATION
```

RecommendationPriority

```
    HIGH

    MEDIUM

    LOW
```

---

12. Final ER Diagram



🎯 Design Decisions

Why only 8 tables?

Because:

- Simpler backend.
- Easier SQLAlchemy models.
- Faster development.
- Meets Friday deadline.
- Easy to extend later.

Why PostgreSQL?

- JSONB support.
- UUID support.
- Excellent SQLAlchemy integration.
- Industry standard.

Why filesystem storage?

- Better performance.
- Smaller database.
- Easier backup.
- Handles large datasets efficiently.

🚨 One Important Architectural Improvement (Highly Recommended)

Veere, eh schema **already production-inspired** aa, but main ik hor improvement suggest karaanga jo future-proof vi aa te MVP nu vi hurt nahi karega.

Instead of putting every file path directly inside each table, assi ik dedicated `artifacts` table bana sakde haan.

Example:

```
artifacts

id

dataset_id

artifact_type
(RAW_FILE,
 CLEANED_FILE,
 MODEL,
 REPORT,
 CHART)
```

```
    file_path

    file_size
```

Then:

- `datasets` only stores dataset metadata.
- Reports, cleaned CSVs, trained models, exported charts—all become artifacts.

Benefits:

- No duplicate file-path columns.
- Future cloud storage (AWS S3, Azure Blob, GCS) becomes much easier.
- Cleaner schema.
- More scalable architecture.
- Still only **9 tables**, which is perfectly reasonable.